

STREAMS

La finalidad de los streams es trabajar con listados de datos.

Comprender⇒ Los **streams** son flujos donde pasaran datos donde le aplicaremos diferentes acciones y obtenerlos al final con el proceso aplicado.

Importante ⇒ aunque trabajemos con la lista la original no es afectada.

Streams son operaciones Terminales

Para que el **flujo funcione correctamente** debemos tener una operación terminal que nos indique cómo lo vamos a **sacar**.

Aquí el concepto está en: *Los streams pueden tener montones de operaciones intermedias perooooo necesitamos SIEMPRE una al final.*

Único uso ⇒ estos no van a poder reutilizarse, lo que quiere decir que se usa y cierra.

SINTAXIS DE LOS STREAMS

(Listado de datos).**stream()**

```
listado.stream()
```

“Es importante comprobar que el listado de mapas puede trabajar con el **stream**”.

MÉTODOS PRINCIPALES INTERMEDIOS

Distintos métodos para hacer estas acciones intermedias que se pueden hacer sobre un stream:

1. **distinct**: elimina elementos duplicados en los streams (Basado en equals())
2. **sorted**: ordena los elementos del stream (como cuando usamos comparador u orden natural)
3. **peek**: permite imprimir lo que ocurre mientras pasan por el stream (utilizado para depuración)
4. **limit**: toma los primeros **n elementos** del listado (relacionado como SQL) según el orden que tiene.
5. **skip**: salta los primeros **n elementos** del listado (viceversa al anterior)
6. **filter**: filtra elementos hay que pasarle un objeto **lambda predicate** (devuelve booleano).

<code>Stream<T></code>	<code>filter(Predicate<? super T> predicate)</code>
------------------------------	---

Que significa el Interrogante del Super T>:

Hasta ahora habíamos trabajado con <T> que eran los tipos, pues en este caso tendrá el mismo tipo o **todas aquellas clases** que estén por encima de esa clase (sus superclases). Aquí buscamos trabajar de la forma más genérica.

Conclusion ⇒ Tipo T o superclase de T

7. **map**: transforma para variar/transformar cada elemento usando otra función. Esta es parecida a filter ya que utiliza también **sintaxis lambda**.
8. **mapToInt / mapToDoble**: se encarga de transformar listas a tipos de datos para trabajar con ellas. Y suele utilizarse para cálculos.

MÉTODOS PRINCIPALES TERMINALES

Como dijimos para que puedan llevarse a cabo debe haber un método terminal, encontramos:

1. **forEach**: recorre todos los elementos del listado de datos del stream, utilizado para mostrar datos (como sysout)
2. **collect**: coge los elementos tras los filtros y nos los da en forma de lista.
3. **collect(Collectors.joining())**: variante de collect. Devuelve una variable de forma estructurada.
4. **count**: devuelve un tipo **long** y básicamente cuenta la cantidad de elementos.
5. **anyMatch**: exige que al menos uno cumpla la condición.
6. **allMatch**: exige que TODOS cumplan la condición.
7. **findFirst()**: buscar el primer elemento específico
8. **reduce**: en este cogemos todo lo que queda de un stream y lo combinamos con el fin de que nos devuelva **un SOLO VALOR**. Ejemplo (El total de un carrito) (La A será el acumulador, y el b será el valor)
9. **min/max**: busca el elemento más pequeño (min) o más grande (máx) según los criterios del comparador.

OPERADOR REFERENCIA:

Lo consideramos como un atajo sintáctico que se enfoca en la **referencia a métodos (::)**. Su principal finalidad es tomar unos parámetros y pasarlo a un método existente.

Como referencias puede tomar:

1. Métodos estáticos
2. instancias de objetos
3. Instancia de un tipo arbitrario.
4. Constructor

QUÉ SON LOS OPTIONALS

Son herramientas que nos ayudan cuando carecemos de valor, un ejemplo con los streams es cuando filtramos y no devuelve nada. (Anteriormente devolvemos null)

Este devolverá un **NullPointerException()** sin embargo aquí con optional evitamos este fallo.

Forma de pensar: debemos verlo como una caja (Optional <t>) cuando pedimos en internet puede (llegar o no llegar). Aquí entramos en el caso de que no llegue (un **null**).

Aquí con el opcional enviamos como mínimo **una caja** (aunque esté vacía) por lo que mínimo nos llegaría una respuesta de donde debería estar (y eliminamos la posibilidad del error)

Entender analogía papel: cuando devolvemos **null** no devolvemos nada, y ahora con **optional** aunque hagamos una encuesta y no haya nadie tendríamos esa lista **vacía**.

MÉTODOS RELEVANTES

1. **.offNullable (T Valor)**: la más importante de todas:
 - a. Crea la caja
 - b. Si el valor existe lo guarda y si no devuelve la caja vacía (no es lo mismo que null).
2. **.isPresent()**: devuelve un True si hay algo sino devolverá un False (**comprobar**)
3. **.ifPresent()**: Aquí introducimos con el uso de los lambdas y sería utilizarlo con el consumer.
4. **orElse()**: es como un plan B donde decimos que si está vacía la caja le daremos otro contenido
5. **orElseThrow()**: aquí metemos excepciones y es si la **caja** está vacía enviará un error. Se usa para lanzar un fallo si el dato no existe.
6. **.map()**: con ello buscamos mapear (*que es transformar*). Aquí básicamente si hay algo dentro lo transforma.
7. **.empty()**: básicamente en vez de poner un **null** si no hallamos nada devolveremos una **caja vacía**. (Evitamos el error).